Getting Started with pyAndorSDK3

Information

Python Wrapper for Andor SDK3
Contains wrapper interface and SDK3 libraries
Supported platforms: Python3.5.1 +

Installation

Installation depending on your python installation:

Open command console within the same directory as the setup.py: NOTE: Using pip to install is preferred

- python3 –m pip install .

or

- python3 setup.py install

Extra functionality to save images as fits files can be included by installing the extra package "save":

- python3 -m pip install .[save]

Extra functionality to display images can be included by installing the extra package "show":

- python3 -m pip install .[show]

Also:

- pip3 install .
- pip3 list
- pip3 uninstall pyAndorSDK3

'sudo' as necessary for Linux

Any errors or suggestions, please report.

Libraries

Windows:

During installation the appropriate libraries (64/32 bit) are copied into the site-packages folder of your python installation.

When the pyAndorSDK3 module is imported the site-packages folder containing the libs is added to the systems PATH variable. To manually specify a location to search for libraries use:

```
from pyAndorSDK3 import utils

utils.add_library_path(r'path\to\directory\containing\libraries')
```

Linux:

The pyAndorSDK3 for Linux assumes the user has installed AndorSDK3 for Linux on their system. This will have correctly configured LD_LIBRARY_PATH as well as configuring the correct symbolic links.

The environment variable LD_LIBRARY_PATH controls the behaviour of the dynamic loader used to find and load the shared libraries needed by a program and so utils.add_library_path() is not used in Linux and will raise NotImplementedError.

Example Initialise and Open

Include and initialise AndorSDK3:

```
from pyAndorSDK3 include AndorSDK3
sdk3 = pyAndorSDK3()
```

There are three ways to retrieve cameras:

1 - Open and retrieve specific camera by index (e.g. for camera at index zero):

```
cam = sdk3.GetCamera(0)
```

2 - Open and retrieve a list of all cameras on the system:

```
cameras = sdk3.cameras
cam1 = cameras[0]
cam2 = cameras[1]
```

3 - Using the 'with' keyword:

```
with sdk3.GetCamera(0) as cam:
    print(cam.SerialNumber)
```

For all above it is important to know it is not necessary to manually call open or close methods. The camera object is automatically opened with GetCamera and is automatically closed when it goes out of scope.

pyAndorSDK3 Class Methods

The pyAndorSDK3() object has a few methods and properties that are of use as described in the table below:

Example Code	Description
sdk3 = pyAndorSDK3()	Initialising the pyAndorSDK3 object and the SDK3 library (analogous to AT_Initialise() SDK3 function)
sdk3.Reinitialise()	Finalises and Reinitialises the SDK3 Library
sdk3.cameras	(Property) a list of cameras opened and available for use
sdk3.DeviceCount	(Property) the number of cameras available
sdk3.SoftwareVersion	(Property) the SDK3 Software Version
sdk3.GetCamera(camera_index)	Retrieves the camera at the user indicated index (if available)

sdk3.event_callback(cb_func)	Creates an event callback function to be used with callback registering. Best used as a decorator (see Callbacks section for more info)

Camera Object Usage

Feature Usage

Using pyAndorSDK3 it is not necessary to call specific typed methods and the knowlegde of the type is figured and handled by the wrapper. It is very simple to set and get features.

Example Code	Description
val = cam.FeatureName	Gets the value of FeatureName (use the SDK3 Feature Matrix for Feature Names)
cam.FeatureName = val	Sets the FeatureName to the value held by the "val" variable
val = cam.CmdFeatureName()	Executes the Command Feature with the name CmdFeatureName
val = cam.max_FeatureName	Gets the maximum value of FeatureName
val = cam.min_FeatureName	Gets the minimum value of FeatureName
opts = cam.options_EnumFeatureName	Gets the list of legal options for the EnumFeatureName feature
avail = cam.is_available_EnumFeatureName("EnumEntry")	Gets the availablity of a given EnumEntry for an EnumFeature (an EnumEntry maybe legal but unavailable at certain times)
avail_opts = cam.available_options_EnumFeatureName	Gets the list of available options for the EnumFeatureName feature
feat_type = cam.type_FeatureName	Gets the feature type of FeatureName as a string

Camera Class Methods

Below are the methods available for use with the Camera class. Most of the available functions revolve around setting up and acquiring images.

Example Code	Description
cam.open()	Opens the camera. NOTE it is not required to call this as the camera is opened automatically If you have manually called close you can call this to reopen the camera associated with this object
cam.close()	Closes the camera instance. NOTE: it is not required to call this method - close is called automaticaly when the object goes out of scope
cam.queue(buffer, buffer_size)	Calls AT_QueueBuffer with a user created buffer

cam.queue_buffer(buffer_size)	Creates and queues an empty buffer of 'buffer_size'
img = cam.wait_buffer(timeout_ms)	Calls the AT_WaitBuffer command and waits for and returns the next available image within the timout period given
cam.flush()	Calls the AT_Flush command and cleans any existing queued buffers
updated_list = cam.get_updated_features()	Returns a list of strings with the names of features that have been updated
cam.register_feature_callback(feature, func)	Register a callback function to a feature (see Callbacks section for more information)
cam.unregister_feature_callback(feature, func)	Unregister a feature callback function (see Callbacks section for more information)
img = cam.acquire()	Acquires a single image - returns an Acquisition object (see Acquisition Object section for more information)
imgs = cam.acquire_series()	Acquires <FrameCount> images and returns a list of Acquisition objects - it is recommended to read the docs for this function below
imgs = cam.get_previous_acquisition_series()	Returns the list of images from the previous acquire_series (useful for retrieval of images when acquire_series failed with an exception)
lib = cam.lib	(Property) Returns an instance of the andor_internals object (still recommended to use the camera through its own interface)
hndl = cam.handle	(Property) The handle of the camera this object owns
index = cam.index	(Property) The index of the camera this object owns

acquire() and acquire_series()

With both the acquire and acquire_series it is possible to pass in features and values you wish to assign for the acquisition(s), and other keyword values for the acquisition. E.g.:

- img = cam.acquire(("ElectronicShutteringMode","Rolling"), timeout=1000)
- imgs = cam.acquire_series(("CycleMode","Fixed"),("FrameCount",100),timeout=5000, max_buf=10, circ_buf=True)

Optional keyword parameters for acquire_series:

Keyword	Type	Default	Description
timeout	int	max(5000, ceil(5000 / cam .FrameRate))	Sets the timeout of each image
min_buf	int	2	Sets the minimum num buffers to be queued
max_buf	int	25	The maximum number of buffers to be assigned before acquisition begins (this is also the muber of buffers used when circular buffer is enabled)

circ_buf	bool	False	Use the assigned buffers in a circular fashion (normal running will create a new buffer for each new acquisition)
frame_count	int	FrameCount feature or 0	The number of frames you want to acquire. Sets FrameCount Feature when in Fixed mode. When this value is not explicitly passed in it uses the FrameCount Feature value as default value when in Fixed CycleMode and 0 as default when using Continuous so will acquire forever - it is advised to use this parameter when using Continuous or use a threaded stop
print_frame	bool	False	Print the data in the acquired frames as they are acquired
print_fps	bool	False	Enable to print Effective FrameRate during the acquisition series
print_fps_interval	int	1	The number of frames to acquire before printing Effective FrameRate
pause_after	float	0.0	Pause for the specified num seconds after all acquisitions are acquirired

The acquire and acquire_series methods can used as a basis to create custom acquisition functions and routines along with the examples custom_acquisition.py and custom_acquisition_circular_buffer.py

Acquisition Object

The Acquisition objects can be interected with in the following ways:

Example Code	Description
img.image	Returns a numpy array of the image data acquired
img.raw_data	Returns a copy of the raw image data acquired
imgs[2].save("/path/to/file/filename", overwrite_if_exist=True)	Saves the second image in a series to path '/path/to/file' and file name "filename.fits" Set optional param 'overwrite_if_exist' to True to force overwrite files)
img.show(cmap="Greys_r", image_name=None)	Displays the image using matplotlib First optional parameter for matplotlib cmap (defaults to Greys_r see matplotlib colormap doc for other options) Second optional parameter for titling the image

Image Metadata

TimeStamp Metadata

MetdataTimeStamp Options	Description

val = img.metadata.timestamp	Returns the value of MetadataTimestamp

Frame Info Metadata

MetatadaFrameInfo Options	Description
val = img.metadata.width	Returns value of width from MetadataFrameInfo
val = img.metadata.height	Returns value of height from MetadataFrameInfo
val = img.metadata.stride	Returns value of stride from MetadataFrameInfo
val = img.metadata.pixelencoding	Returns value of pixelencoding from MetadataFrameInfo

IRIG Metadata

MetatadaIRIGB Options	Description
val = img.metadata.irig_nanoseconds	Returns value of nanoseconds from MetadataIRIGB
val = img.metadata.irig_seconds	Returns value of seconds from MetadataIRIGB
val = img.metadata.irig_minutes	Returns value of minutes from MetadataIRIGB
val = img.metadata.irig_hours	Returns value of hours from MetadataIRIGB
val = img.metadata.irig_days	Returns value of days from MetadataIRIGB
val = img.metadata.irig_years	Returns value of years from MetadataIRIGB

Callbacks

Callbacks allow the application to receive notification when a feature has indirect change occur, a callback function can be created and attached to a feature. Whenever the feature changes in any way, this callback will be triggered, allowing the application to carry out any actions required to respond to the change.

A callback should complete any work required in the minimal amount of time as it holds up the thread that caused the callback. If possible the application should delegate any work to a separate application thread if the action will take a significant amount of time. The callback function should not attempt to modify the value of any feature as this can cause lockup

Registering callbacks for feature updates is a simple process with 2 main steps. First is creating the callback function. The second is registering that function to a feature. On successfully registering a callback to a feature the callback function will execute once.

```
@sdk3.event_callback    # notice using sdk.event_callback method as a decorator
def func(handle, feature):
    print("callback handle: {}  feature: {}  value: {}".format(handle, feature, getattr(cam, feature)))

cam.register_feature_callback("ExposureTime", func)
```

It is not necessary to manually unregister feature callbacks as it is done automatically when the camera object goes out of scope. But this can be done as below:

```
cam.unregister_feature_callback("ExposureTime", func)
```

For a fully executable example see the example callback.py.

Examples

Below is the list of examples available and a small description:

Example Name	Description
get_serial_number.py	Simply showing how to open the camera and retrieve the camera's SerialNumber.
get_image_data.py	Demonstrating the use of cam.acquire() and cam.acquire_series() methods and showing how the frame_count keyword interacts with the CycleMode.
get_image_data_circular_buffer.py	Using cam.acquire_series() with the circ_buf keyword.
custom_single_acquisition.py	Starting acquisition and acquiring a single image. This is the most basic acquisition example showing the minimum requirement for acquiring.
custom_acquisition.py	An example acquisition for acquiring a series of images.
custom_acquisition_circular_buffer.py	An example acquisition for acquiring a series of images and requeuing buffers in a circular fashion.
get_metadata.py	Shows how to retrieve metadata from Acquisition objects.
callbacks.py	The creation and registering of callback functions to features.
using_with_statement.py	Showing opening and using the camera using a with statement.
error_handling.py	Contains various examples of catching CameraExceptions thrown by pyAndorSDK3 and using the ErrorCodes enum to handle.
multi_camera.py	Demonstrating how to open and control multiple cameras at once.
framerate_max.py	Example of how the framerate by the camera can produce data faster than the interface can trnasfer and how to handle this situation.
sensor_cooling.py	Setting a camera sensor target temperature and waiting for temperature to stabilise.
accumulation.py	An couple of examples to show how to use Image Accumulation.
save_images.py	Examples for saving image data using acq.save() and astropy.
show_image.py	Example for using the acq.show() method.

For SDK3 usage or feature specific information please refer to the manual Andor Software Development Kit 3.pdf